

Shri. R. K. Parikh Arts & Science College, Petlad

B.Sc (Computer Science) – Semester – 4

Unit: 1 Structures and Unions

❖ Basics of Structures, Structures and functions, Structures and Arrays :

▲ **Structure**

- A structure is a collection of related data elements of the same or different data types.
- The data elements inside structure are known as members.

Structure declaration:-

- The syntax for declaring structure in C is shown below.

```
struct    struct_tag
{
    datatype    member1 ;
    datatype    member2;
    :
    :
};
```

- The structure declaration starts with the keyword struct follow by a structure tag.
- The structure tag is used for creating structure variables.
- The individual members of the structure are inside the curly braces.
- The closing brace is terminated with a semicolon.

Example:-

- Consider a student database consisting of student roll number, name, branch and total marks scored. A structure declaration for this definition is given below.

```
struct    student
{
    int      roll_no ;
    char     name[50] ;
    char     branch[20] ;
    int      marks ;
};
```

Structure definition:-

- The structure definition creates structure variables and allocate storage space for them.
- Structure variables can be created.
 - 1) **At the point of structure declaration itself or**
 - 2) **Any where in program below structure declaration by using structure tag.**

- The structure variables can be created during the declaration of a structure as below.

```
struct student
{
    int    roll_no ;
    char   name[50] ;
    char   branch[20] ;
    int    marks ;
} s1;
```

- In above declaration, student is the structure tag and 's1' is a variable of structure student.
- It is also possible to declare structure variable without using structure tag.

```
struct
{
    int    roll_no ;
    char   name[50] ;
    char   branch[20] ;
    int    marks ;
} s1;
```

- In above declaration, structure tag is omitted and 's1' is a variable of structure student.
- Another way for creating structure variable is as follows.

Syntax:-

struct struct_tag var1, var2, ... varN ;

- The keyword struct is optional here.

struct student s;
OR
student s;

Accessing structure members:-

1. The dot operator (.) is used to access the structure members.
2. The link between a member of a structure and variable of a structure is established by dot operator.
3. The dot operator is also known as Member operator OR Period operator.
4. The syntax for accessing the structure member is shown below.

StructVar.MemberName

- Here, StructVar is a structure variable and MemberName is member name.

For example:-

s.roll_no = 5 ;
or

```
printf("Enter student roll number...");
scanf("%d", &s.roll_no);
```

- Note that accessing members of a structure using structure tag is not allowed.
- Hence, the following are invalid statements.

```
Student.roll_no = 5 ;
```

or

```
printf("Enter student roll number...");
scanf("%d", &student.roll_no);
```

Structure Initialization:-

- A structure variable can be initialized at compile time, as same as initialization of variable of other data type.
- A lot of variation is possible in initializing the variables of a structure.
- The initialization must be done only in the declaration of the structure variable.
- One way to initialize a structure variable Outside the main() function as shown below:

```
struct student
{
    int    roll_no, marks ;
    char   name[50] ;
    char   branch[20] ;
} s = {1, "Ekta Kapoor", "Computer", 300};
void main()
{
    :
    :
}
```

OR

- Another way to initialize a structure variable inside the main() function as shown below:

```
struct student
{
    int    roll_no ;
    char   name[50] ;
    char   branch[20] ;
    int    marks ;
} s ;
void main()
{
    student s = {1, "Ekta Kapoor", "Computer", 300};
    : }
}
```

- The members of a structure, roll_no, name, branch and marks are initialized with 1, Ekta Kapoor, Computer and 300 respectively.
 - The individual members cannot be initialized inside the structure declaration. For example, the following declaration is invalid.
- ```
struct student
```

```

{
 int roll_no = 1 ; // initialization not allowed here
 char name[50] ;
 char branch[20] ;
 int marks ;
};

```

### ❖ Nesting of Structure:-

- A structure declared inside another structure is known as nesting of structure.
- It is also known as Structure within Structure.

#### For example:-

- One way of declaring nested structure is shown below.

```

struct date
{
 int day, month, year;
};
struct student
{
 int roll_no ;
 char name[50] ;
 struct date birthday;
 char branch[20] ;
 int marks ;
};

```

OR

Another way of declaring a nested structure is as below.

```

struct student
{
 int roll_no ;
 char name[50] ;
 struct date
 {
 int day, month, year;
 } birthday;
 char branch[20] ;
 int marks ;
} S1 ;

```

- In above example, the members of a structure are
  - Roll\_no – Roll number of a student
  - Name – Name of a student
  - Birthday – It is itself a structure with three members.
  - Branch – Student branch
  - Marks – Total marks of a student

- The members contained in the student are roll\_no, name, branch and marks can be accessed as

S1.roll\_no  
S1.name  
S1.branch  
S1.marks

- The members contained in the birthday are day, month and year can be accessed as  
S1.birthday.day  
S1.birthday.month  
S1.birthday.year

### **Array within Structure:-**

- It is possible to declare an array as the members of a structure. Consider the following structure definition.

```
struct student
{
 int no;
 char name[25];
 int marks[6];
};
```

- In above, "name" and "marks" are an array declared inside a structure student. They are one-dimensional array. We can also declare multi-dimensional array inside of a structure.
- The array "marks" contains marks of six subjects. The individual subject marks represented as marks[0], marks[1], .....marks[5].

**Program : Build a structure student having student number, name and marks of six subject. Write a C program to read and print student details using structure.**

```
#include<stdio.h>
#include<conio.h>
struct student
{
 int sno;
 char sname[50];
 int marks[6];
};
void main()
{
 struct student s1;
 int i;
 clrscr();
 printf("Enter number :");
 scanf("%d",&s1.sno);
```

```

printf("Enter name :");
fflush(stdin);
gets(s1.sname);
for(i=0; i<6 ; i++)
{
 printf("Enter marks :");
 scanf("%d",&s1.marks[i]);
}
printf("\nStudent Result");
printf("\nNumber : %d",s1.sno);
printf("\nName : %s",s1.sname);
for(i=0;i<6;i++) printf("\nSubject-%d : %d",i+1,s1.marks[i]);
getch();
}

```

### **Input & Output:-**

Enter number : 1  
 Enter name : a b shah  
 Enter marks : 10  
 Enter marks : 20  
 Enter marks : 30  
 Enter marks : 40  
 Enter marks : 50  
 Enter marks : 60  
 Student Result  
 Number : 1  
 Name : a b shah  
 Subject-1 : 10  
 Subject-2 : 20  
 Subject-3 : 30  
 Subject-4 : 40  
 Subject-5 : 50  
 Subject-6 : 60

### **Array of Structure:-**

- It is possible to define an array of structure variable. Such a variable is known as array of structure.
- Consider the following structure definition.

```

struct student
{
 int no;
 char name[25];
 int marks;
};

```

- The syntax for defining an array of structure is shown below.

```

struct Struct_Name Array_Name [size] ;
struct_Name Array_Name [size];

```

- In above, struct is keyword, Struct\_Name is the name of the structure, Array\_Name is any valid variable name and size is integer constant value. An array of the above structure can be defined as follows.

```
struct student s[50];
or
student s[50];
```

This declares the "s" as an array of the structure.

- The syntax for accessing its members using an index.

**ArrayName [ index ]. MemberName**

- The following statement access members of the structure array elements.

```
scanf("%s", s[4].name); //It reads name of the 5th student.
scanf("%d", &s[2].marks); //It reads marks of the 3rd student.
printf("%d", s[0].no); // It prints number of 1st student.
```

**Program: Following program reads the number, name and total marks of the N number of student and displays them.**

```
#include<stdio.h>
#include<conio.h>
struct student
{
 int no;
 char name[25];
 int marks;
};
void main()
{
 student s[50];
 int i,n;
 clrscr();
 printf("Enter number of student...");
 scanf("%d", &n);
 for(i=0 ; i<n ; i++)
 {
 printf("Student number...");
 scanf("%d", &s[i].no);
 printf("Student name...");
 scanf("%s", s[i].name);
 printf("Student total marks...");
 scanf("%d", &s[i].marks);
 }
 printf("\n Student Report ");
 printf("\n -----");
 printf("\n No Name Marks");
 printf("\n -----");
 for (i=0 ; i<n ; i++)
```

```

 printf("\n %d %s %d ",s[i].no,s[i].name,s[i].marks);
 getch();
}

```

Input:

```

Enter number of student... 2
Student number...1
Student name...Anjali
Student marks...50
Student number...2
Student name...Tushar
Student marks...80

```

Output:

```

Student Report

No. Name Marks

1 Anjali 50
2 Tushar 80

```

**Question : Can entire structure variable be assigned to another structure variable, provided both variables having same structure? Answer with example.**

Answer :

- Yes, it is possible to assign one structure variable to another structure variable. But in the case the structure of both the variables are same.
- Like a normal variable, a structure variable of same structure type can be copied from one variable to other variable. Consider the following example:

```

struct stud
{
 int sno;
 char sname[50];
} s1, s2;

```

- In above example, s1 and s2 are structure variable of same structure. The following statement is valid.
 

```
s1 = s2 ;
```
- It will copy the contents of s2 into s1. Similarly the following statement is also valid.
 

```
s2 = s1 ;
```
- It will copy the contents of s1 into s2.

### **Structure & Function :**

**Question : Can entire structure variable be passed to a function as an argument?**

**Answer with example.**

Answer :

- Yes, it is possible to passed an entire structure variable to a function as an argument.
- Following program demonstrates how to pass a structure variable to a function as an argument.

```

#include<stdio.h>
#include<conio.h>
struct book
{
 int bno;
 char bname[50];
};
void print(struct book t)
{
 printf("\nBook Number : ",t.bno);
 printf("\nBook Name : %s",t.bname);
}
void main()
{
 struct book b1={100,"C Programming"};
 clrscr();
 print(b1);
 getch();
}

```

Output:  
 Book Number : 100  
 Book Name : C Programming

- In above program, the statement print(b1) called the print() function declared as outside of a main() function.
- This statement contains a variable b1 of structure type as a parameter. The contents of variable b1 passed to the function print(struct book t). It will display the details of a book.

### ❖ Pointer and Structure:

- The structure variable is the address of its zeroth element. Consider the following student structure.

```

struct student
{
 int roll_no;
 char name[50];
} s[5], *ptr;

```

- The following statement assigns the address of the student structure into pointer variable. ptr = s;
- Now, the pointer ptr contains the address of s[0]. Its members can be access using following statement.  
 ptr -> roll\_no  
 ptr -> name
- The symbol -> is known as arrow operator. Arrow operator is made up of a minus sign and a greater than sign. When the pointer ptr is incremented by one, it is made to point to the next record, i.e s[1].

- Another way to access element using dot operator  
(\*ptr).roll\_no  
(\*ptr).name

**Program: Write a C program that display use of pointer to access the members of structure.**

```
#include<stdio.h>
#include<conio.h>
struct student
{
 int roll_no;
 char name[50];
};
void main()
{
 student s[10] = { { 1, "abc" },
 { 2, "xyz" },
 { 3, "pqr" }
 };
 student *ptr;
 int n=3;
 clrscr();
 ptr = s;
 i = 0;
 while (i < 3)
 {
 printf("\n %d %s", ptr->roll_no,ptr->name);
 ptr ++;
 }
 getch();
}
```

**Output:**

```
1 abc
2 xyz
3 pqr
```

## ❖ Unions, Working and initializing with unions:-

### Union

- Union can be defined as a user-defined data type which is a collection of different variables of different data types in the same memory location.
- The union can also be defined as many members, but only one member can contain a value at a particular point in time.
- Union is a user-defined data type, but unlike structures, they share the same memory location.

The union is declared as follows.

```
union union_tag
```

```

{
 datatype1 member1;
 datatype2 member2;
 : :
 : :
 datatypeN memberN;
};

```

- A union is same as structure except its storage space requirement.
- All the member of a union share common memory space.
- The amount of memory required is always equal to that required by its largest member.

**The amount of memory required = max ( sizeof(member1) + sizeof(member2) + .. sizeof(memberN) ).**

**Note :**

- Size of the union is the size of its largest field because sufficient number of bytes must be reserved to store the largest sized field.
- To access the fields of a union, use dot(.) operator i.e., the variable name followed by dot operator followed by field name.
- To access pointer variables, we use the -> operator.

**Example:**

```

union temp
{
 int i; // 2 bytes
 char c; // 1 byte
 float f; // 4 bytes
} union_var;

```

The total storage required for above union variable union\_var is of size 4 bytes.

```

#include <stdio.h>
#include <string.h>

```

```

union Data {
 int i;
 float f;
 char str[20];
};

int main() {

 union Data data;

 printf("Memory size occupied by data : %d\n", sizeof(data));
}

```

```

 return 0;
}

```

Memory size occupied by data : 20

```

#include <stdio.h>
#include <string.h>

```

```

union Data
{
 int i;
 float f;
 char str[20];
};

```

```

int main()
{
 union Data data;

 data.i = 10;
 printf("data.i : %d\n", data.i);

 data.f = 220.5;
 printf("data.f : %f\n", data.f);

 strcpy(data.str, "C Programming");
 printf("data.str : %s\n", data.str);

 return 0;
}

```

```

data.i : 10
data.f : 220.500000
data.str : C Programming

```

Here, all the members are getting printed very well because one member is being used at a time.

## ❖ Structures versus Unions:-

### Differentiate between Structure and Union.

|    | Structure                                                                                                   |    | Union                                                              |
|----|-------------------------------------------------------------------------------------------------------------|----|--------------------------------------------------------------------|
| 1. | A structure is collection of logical related data items of same or different data types into a single unit. | 1. | A union is same as structure except its storage space requirement. |
| 2. | The structure is declared as follows.                                                                       | 2. | The union is declared as follows.                                  |

|    |                                                                                                                                                                                         |    |                                                                                                                                                                                      |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <pre> struct    struct_tag {     datatype1    member1;     datatype2    member2;         :        :     datatypeN    memberN; }; </pre>                                                 |    | <pre> union    union_tag {     datatype1    member1;     datatype2    member2;         :        :         :        :     datatypeN    memberN; }; </pre>                             |
| 3. | The amount of memory required to store a structure variable is the sum of the size of all the members.                                                                                  | 3. | The amount of memory required is always equal to that required by its largest member.                                                                                                |
| 4. | The amount of memory required = sizeof(member1) + sizeof(member2) + . . . . . sizeof(memberN).                                                                                          | 4. | The amount of memory required = max ( sizeof(member1) + sizeof(member2) + . . . . . sizeof(memberN) ).                                                                               |
| 5. | All the member of a structure can be accessed at any given time.                                                                                                                        | 5. | Only one member of a union can be accessed at any given time because only one variable is active.                                                                                    |
| 6. | All the member of a structure has different memory space.                                                                                                                               | 6. | All the member of a union share common memory space.                                                                                                                                 |
| 7. | <p>Example:</p> <pre> struct temp {     int i;     char c;     float f; } struct_var ; </pre> <p>The total storage required for above union variable struct_var is of size 7 bytes.</p> |    | <p>Example:</p> <pre> union temp {     int i;     char c;     float f; } union_var ; </pre> <p>The total storage required for above union variable union_var is of size 4 bytes.</p> |

### ❖ Typedef and enum keyword:-

#### Defined data types (typedef):-

- The C language allows the definition of our own types based on other existing data types. We can do this using the keyword typedef.
- The general form of typedef is:

```
typedef existing_type new_type_name ;
```
- where existing\_type is a fundamental data type of C language, new\_type\_name is the name for the new type

#### **Examples:**

```

typedef char mychar;
typedef int INT;

```

- In this case we have defined two new data types: mychar and INT as char and integer respectively. It only creates synonyms of existing types. Now, the following statement  
mychar choice;  
INT jawab;
- It means that the variable choice can be considered to be either mychar or char, since both are in fact the same type.
- Therefore, typedef can be useful to define an alias for a data type that is frequently used within a program.

### **Enumeration keyword:-**

- Enumeration is a user defined datatype in C language. It is used to assign names to the integral constants which makes a program easy to read and maintain. The keyword "enum" is used to declare an enumeration.
- Here is the syntax of enum in C language,  
enum enum\_name{const1, const2, ..... };
- The enum keyword is also used to define the variables of enum type. There are two ways to define the variables of enum type as follows.  
enum week{sunday, monday, tuesday, wednesday, thursday, friday, saturday};  
enum week day;

Example

```
#include<stdio.h>
enum week{Mon=10, Tue, Wed, Thur, Fri=10, Sat=16, Sun};
enum day{Mond, Tues, Wedn, Thurs, Frid=18, Satu=11, Sund};
void main()
{
printf("The value of enum week: %d\t%d\t%d\t%d\t%d\t%d\t%d\n\n", Mon ,
Tue, Wed, Thur, Fri, Sat, Sun);
printf("The default value of enum day: %d\t%d\t%d\t%d\t%d\t%d\t%d", Mond ,
Tues, Wedn, Thurs, Frid, Satu, Sund);
}
```

Output

The value of enum week:

10    11    12    13    10    16    17

The default value of enum day:

0    1    2    3    18    11    12

# Shri. R. K. Parikh Arts & Science College Petlad

## US04CCSC51- Advanced C Programming and Introduction to Data Structures

### QUESTION BANK

#### UNIT -1 (MCQ)

- 1 Given a structure variable named stu which is of type struct STU and contains a field named major, which of the following statements correctly refers to major?  
A. Major      B. stu-major      **C. stu.major**      D. STU.major
- 2 Which of the following statement is not true about structure?  
A. A field in a structure can itself be a structure.  
B. Individual members of a structure can be passed to the function.  
C. Structure definition is just a template which is used to declare variables of type struct.  
**D. A structure can not have two fields with same type.**
- 3 Given a pointer ptr to a structure stu containing a field called name which of the following statements correctly refer name?  
**A. ptr->name**      B. ptr.name      C. ptr->stu.name      D. ptr->stu->name
- 4 Which of the following is not a derived data type?  
A. Arrays      **B. Float**      C. Pointers      D. Structure
- 5 Which of the following allows a portion of memory to be shared by different types of data?  
A. Array      B. Structure      **C. Union**      D. File
- 6 Determine which of the following statement is true?  
A. A union can have another union as one of the fields.  
B. When accessing the fields of a structure through a pointer p, (\*p).field\_name and p->field\_name both are same.  
C. A structure can have another structure as one of the fields.  
**D. A structure can not have an array as one of its elements.**
- 7 Determine which of the following statement is false?  
A. A structure can be initialized when it is defined.  
**B. The following two expressions are same if ptr is a pointer to structure:  
\*p.x and p->x**  
C. We can access the members of a structure using member operator  
D. We can access the members of a structure through a pointer using member selection operator.

- 8 Which of the following statement is true for structures?
- A. A structure can be copied to another structure of same type using assignment operator.**
  - B. Structure definition that does not contain tag can be used in future to create variables of type structure.
  - C. If ptr is a pointer to structure student then ptr->filed\_name is used to refer a member of structure student.
  - D. Definition of structure variable does not reserve space in memory.
- 9 Which of the following method is not valid to send information in a structure to a function?
- A. Pass each member of the structure as an actual argument
  - B. Pass copy of entire structure
  - C. Pass structure definition**
  - D. Pass address of structure
- 10 Which of the following can be used to create a new type that can be used anywhere a type is permitted?
- A. Typedef**
  - B. Array
  - C. struct
  - D. Both struct and typedef
- 11 Which of the following statement is false about structure?
- A. Structure can be nested.
  - B. Structure can have many members of same or different data types.
  - C. All elements of array of structure are not created in continuous memory locations.**
  - D. Array of structure ca be created.
- 12 Consider following declaration:
- ```
struct employee
{
    int eno;
    char enm[20];
    struct
    {
        int bsal;
        float bonus;
    } salary, sal;
}e1;
```

Which of the following statement is false?

- A. Above declaration is wrong. It will give error.**
- B. We can refer bonus with e1.salary.bonus as well as e1.sal.bonus.
- C. We can refer eno with e1.sal.
- D. salary and sal both can be used to refer members of inner structure.

- 13 Which of the following statement is true about union?
- A. We can initialize all members of union.
 - B. If you fill in a member of one type and then try to use a different type, you can get unpredictable results.**
 - C. All union members can not be accessed at the same time.
 - D. Size of a union is equal to total of size of all members.
- 14 Structures can not be initialized at
- A. Run time by taking input from user
 - B. Run time by assignment statement
 - C. Compile time at time of declaration of structure variable
 - D. Compile time within structure definition using assignment statement**
- 15 Which of the following statement is false?
- A. Each variable / data field within a structure is called a member of the structure OR structure elements.
 - B. Definition of structure describes a format called template that can be used later in a program to declare structures.
 - C. Individual structure members can be accessed like other variables because they are like variable.**
 - D. Structure variables are known as an instance.

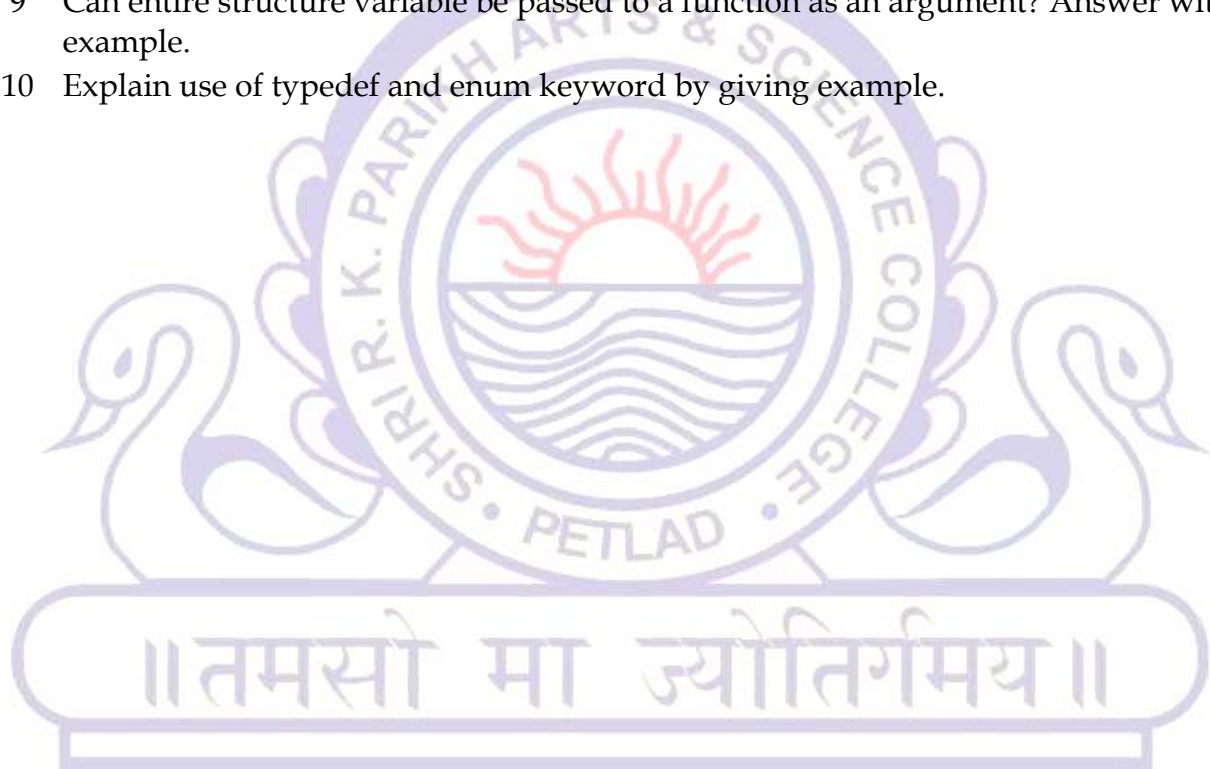
Short Questions

- 1 Define: structure, member operator
- 2 Differentiate: structure and union
- 3 Can entire structure variable be assigned to another structure variable, provided both variables having same structure? Answer with example.
- 4 Can entire structure variable be passed to a function as an argument? Answer with example.
- 5 Define a structure called college which contains an integer quantity called college_code and a string called c_name with length 20. Create structure called courses within structure college which contains integer quantity called course_no and a float quantity called class_result. Create one variable called college1 along with definition.
- 6 Define a union called "item" consisting of an integer called code, character string called nm and float value called price. Declare union variable called product along with definition.
- 7 Explain typedef in brief with suitable example.
- 8 Differentiate '.' and '->' operators.
- 9 Consider ptr is a pointer pointing to struct stud and rollno is a member of struct stud. Mention two methods of accessing member rollno of struct stud through pointer ptr.
- 10 Consider ptr is a pointer pointing to struct stud and rollno is a member of struct stud. We can access member rollno of struct stud through pointer ptr using (*ptr).rollno Why we need to use parenthesis around *ptr?

- 11 What we mean by structure definition? What is the importance of tag in structure definition?
- 12 Explain use of keyword enum in brief.

Long Questions

- 1 What is structure? Explain its definition, declaration and assigning values to members of structure. Also mention advantages of structure over other data types.
- 2 Explain array of structures using suitable examples.
- 3 What is union? Explain its definition, declaration and assigning values to members of union.
- 4 What is union? Explain its storage representation. How a member of union is assigned an initial value? Explain in brief with example.
- 5 Write note on: structure within structure
- 6 Explain array within structure using suitable example.
- 7 Write note on: pointer to structure
- 8 Explain pointer to structure array using appropriate example.
- 9 Can entire structure variable be passed to a function as an argument? Answer with example.
- 10 Explain use of typedef and enum keyword by giving example.



Shri. R. K. Parikh Arts & Science College, Petlad

B.Sc (Computer Science) – Semester – 4

Unit: 2 File Handling

❖ Introduction to File handling and usage:-

Introduction to File Handling:

- The printf() and scanf() are I/O functions. The scanf() function is used to read the data from keyboard and the printf() function is used to print data on the screen. It is good for small volume of data. But for the large volume of data, it has following problems:

Drawbacks (problems) of I/O operation:

- It is difficult and time consuming to handle large volume of data through keyboard.
- A data is lost when the program is terminated or the computer is turned off.
- The concept of file is used, to solve the above problem. A file is a place on the disk where a group of data is stored.

❖ Operations on files, File access modes, Handling text files

Operations on a file:

- The C language supports following operations on a file:
 - Naming a file
 - Opening a file
 - Reading data from a file
 - Writing data to a file
 - Closing a file
- There are two ways to perform file operations in C.
 - Low-level I/O operation
 - High-level I/O operation

The following table gives the list of file handling functions:

Function Name	Operations
fopen()	Creates a new file or opens an existing file.
fclose()	Closes a file.
getc()	Reads a character from a file.
putc()	Writes a character on a file.
fprintf()	Writes a set of data values on a file.
fscanf()	Reads a set of data values from a file.
getw()	Reads an integer from a file.
putw()	Writes an integer on a file.

Defining and opening a file:

- The syntax for declaring a file is as below:
FILE *fp;
- The above statement declares the variable fp as a pointer to the data type FILE. It is defined in the stdio.h file. It is used to define a file pointer for use in file operations.
- Example:
`FILE *fp;`
- The syntax for opening a file is as below:
`fp = fopen("filename", "mode");`
- Before we can read/write a data from a file, we must open it. This can be done by the fopen() function.
- The first parameter is the name of a file. The file name is any valid DOS file name, and can be expressed in upper or lower case letters, or even mixed. It is enclosed in double quotes.
- The second parameter is the mode of a file. It can be any of the three letters, r, w, or a. A mode of a file must be in lower case.

A mode of a file can be one of the following:

- r – Opens the file for reading
- w – Opens the file for writing
- a – Opens the file for appending

- The mode of a file should be inside double quotes.
- A mode of a file must be in lower case.
- Opening a file for reading requires that the file already exist. If it does not exist, the file pointer will be set to NULL.
- Opening a file for writing does not require the existence of a file. If it does not exist, it will be created. If it exists, the contents of a file will be deleted and create as a new file.
- When a file is opened for appending, it will be created if it does not already exist and it will be initially empty. If it does exist, the data input point will be positioned at the end of the present data.

Example-1:

```
FILE *fp;  
fp = fopen("d:\stud.txt", "r");
```

The file stud.txt opens for a reading. If it fails to open then it store NULL value into a file pointer fp.

Example-2:

```
FILE *fp;  
fp = fopen("d:\stud.txt", "w");
```

The file stud.txt opens for a writing.

Example-3:

```
FILE *fp;  
fp = fopen("d:\stud.txt", "a");
```

The file stud.txt opens for an appending.

Closing a file:

- A file must be close after it used is completed.
- The syntax for closing a file is as below:

fclose(file_pointer) ;

- Example:

```
➤ FILE *fp;
  fp = fopen("d:\stud.txt", "r");
  :
  :
  :
  close(fp);
```

- Once a file is closed, its file pointer can be reused for another file. All files are closed automatically whenever a program terminates.

❖ File management I/O functions

Input / Output operation on Files:

- The input/output operations on a file can be done by following functions:
 1. The getc() and putc() functions
 2. The getw() and putw() functions
 3. The fscanf() and fprintf() functions

[1] The getc() and putc() functions:-

getc()

Meaning:

It is used to read a character from a file.

Syntax:

```
int getc(FILE *fp)
```

```
ch. var. = getc(filepointer)
```

Where ch. var is previously declare character type variable, getc() is a function to read single character from file and file pointer is name of pointer variable used in fopen() statement.

Description:

- It is used to read a character from a file.
- It returns EOF, when end of file has been reached.
- A file must be opened with the read mode.
- It moves a file pointer from a current character to next character on every read.
- It handles only one character at a time.

Example:

//To displays contents of a file (stud.txt) on the screen.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```

FILE *fp;
int c;
clrscr();
fp = fopen("d:\stud1.txt","r");
if(fp==NULL)
    printf("File not found...");
else
{
    c=getc(fp);
    while( c!=EOF)
    {
        printf("%c",c);
        c=getc(fp);
    }
}
fclose(fp);
getch();
}

```

putc()

Meaning:

It is used to write a character onto a file.

Syntax:

```

int putc(int c, FILE *fp)
putc(ch.var,file pointer);

```

where putc() is a function used to write character in given file, Ch. var is character type variable which you want to write in file, and file pointer is pointer variable used in fopen() function.

Description:

- It is used to write a character onto a file.
- A file must be opened with the write or append mode.
- It moves a file pointer from a current character to next character on every write.
- It handles only one character at a time.

Example:

// To copy the contents of a file (s1.txt) to another file(s2.txt).

```

#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *f1,*f2;
    int c;
    clrscr();
    f1 = fopen("d:\s1.txt","r");
    f2 = fopen("d:\s2.txt","w");
    if(f1==NULL)
        printf("File not found...");
    else

```

```

    {
        c=getc(f1);
        while( c!=EOF)
        {
            putc(c,f2);
            c=getc(f1);
        }
    }
    fclose(f1);
    fclose(f2);
    printf("File copied...");
    getch();
}

```

[2] The getw() and putw() functions :-

getw()

Meaning:

It is used to read an integer value from a file.

Syntax:

int getw(FILE *fp)

integer var. = getw(filepointer)

Where integer var is previously declare integer type variable, getw() is a function to read integer number from file and file pointer is name of pointer variable used in fopen() statement.

Description:

- It is used to read an integer value from a file.
- It returns EOF, when end of file has been reached or an error occurs.
- A file must be opened with the read mode.
- It should not allow for opening a file in text mode.

Example:

```

#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *f1,*f2;
    int c,i,marks;
    clrscr();
    f1 = fopen("z:\s1.txt","w");
    for(i=1;i<=3;i++)
    {
        printf("Enter total marks:");
        scanf("%d",&marks);
        putw(marks,f1);
    }
    fclose(f1);
    f2 = fopen("z:\s1.txt","r");
    printf("\nYou have entered...");
}

```

```

for(i=1;i<=3;i++)
{
    marks=getw(f2);
    printf("%d ",marks);
}
fclose(f2);
getch(); }

```

putw()

Meaning:

It is used to write an integer value on a file.

Syntax:

putw(int n, FILE *fp)

putw(int.var,file pointer);

where putw() is a function used to write integer number in given file, int. var is integer type variable which you want to write in file, and file pointer is pointer variable used in fopen() function.

Description:

- It is used to write an integer value on a file.
- It returns EOF, an error occurs.
- A file must be opened with the write/append mode.
- It should not allow for opening a file in text mode.

Example:

```

#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *f1,*f2;
    int c,i,marks;
    clrscr();
    f1 = fopen("z:\s1.txt","w");
    for(i=1;i<=3;i++)
    {
        printf("Enter total marks:");
        scanf("%d",&marks);
        putw(marks,f1);
    }
    fclose(f1);
    f2 = fopen("z:\s1.txt","r");
    printf("\nYou have entered...");
    for(i=1;i<=3;i++)
    {
        marks=getw(f2);
        printf("%d ",marks);
    }
    fclose(f2);
    getch();
}

```

[3] The fscanf() and fprintf() functions:

fscanf()

Meaning:

It is used to read a group of mixed data from a file.

Syntax:

fscanf(FILE *fp, "control string", list) ;

where fscanf() is a function used to read mixed data from given file, File pointer is a pointer variable which used in fopen() function, control string contains format specification for variable specified in list and list is list of variables that we want to read from a file.

Description:

- It is used to read a group of mixed data from a file.
- It is same as scanf() function except it works on file.
- It returns EOF, when end of file has been reached or an error occurs.
- A file must be opened with the read mode.

Example:

//Program on fprintf() and fscanf() function

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
    FILE *f1;
```

```
    int i,sno;
```

```
    char sname[10];
```

```
    clrscr();
```

```
    f1 = fopen("z:\s1.txt","w");
```

```
    for(i=1;i<=2;i++)
```

```
    {
```

```
        printf("Enter total marks:");
```

```
        scanf("%d",&sno);
```

```
        printf("Enter name:");
```

```
        fflush(stdin);
```

```
        gets(sname);
```

```
        fprintf(f1,"%d %s ",sno,sname);
```

```
    }
```

```
    fclose(f1);
```

```
    f1 = fopen("z:\s1.txt","r");
```

```
    printf("\nYou have entered...");
```

```
    for(i=1;i<=2;i++)
```

```
    {
```

```
        fscanf(f1,"%d %s",&sno,sname);
```

```
        printf("\n%d %s",sno,sname);
```

```
    }
```

```
    fclose(f1);
```

```
    getch();  
}
```

fprintf()

Meaning: It is used to write a group of mixed data on a file.

Syntax: fprintf(FILE *fp, "control string", list) ;
where fprintf() is a function used to write mixed data into given file, File pointer is a pointer variable which used in fopen() function, control string contains format specification for variable specified in list and list is list of variables that we want to write into a file.

Description:

- It is used to write a group of mixed data on a file.
- It is same as printf() function except it works on file.
- It returns EOF, when an error occurs.
- A file must be opened with the write/append mode.

Example: //Program on fprintf() and fscanf() function

```
#include<stdio.h>  
#include<conio.h>  
void main()  
{  
    FILE *f1;  
    int i,sno;  
    char sname[10];  
    clrscr();  
    f1 = fopen("z:\s1.txt","w");  
    for(i=1;i<=2;i++)  
    {  
        printf("Enter total marks:");  
        scanf("%d",&sno);  
        printf("Enter name:");  
        fflush(stdin);  
        gets(sname);  
        fprintf(f1,"%d %s ",sno,sname);  
    }  
    fclose(f1);  
    f1 = fopen("z:\s1.txt","r");  
    printf("\nYou have entered...");  
    for(i=1;i<=2;i++)  
    {  
        fscanf(f1,"%d %s",&sno,sname);  
        printf("\n%d %s",sno,sname);  
    }  
    fclose(f1);  
    getch();  
}
```

Shri. R. K. Parikh Arts & Science College Petlad

US04CCSC51- Advanced C Programming and Introduction to Data Structures

QUESTION BANK

UNIT -2 (MCQ)

- 1 Which functions are used to write mixed data into a file?
A. **fprintf()** B. putc() C. putw() D. None of these
- 2 What is the function of getc() function?
A. To read a number from file C. To write character in to file
B. **To read a character from file** D. All of the above
- 3 Which file modes are used in 'C'?
A. **"r","w" and "a"** C. "read", "write" and "append"
B. "R", "W" and "A" D. None of these
- 4 Which functions are used to read integer numbers from file?
A. getc() B. **getw()** C. fscanf() D. None of these
- 5 Which functions are used to write integer numbers into a file?
A. fprintf() B. putc() C. **putw()** D. None of these
- 6 What are two predefined FILE pointers in C?
A. stdout and stderr B. **stdout and stdio**
C. console and error D. stdio and stderr
- 7 Code:
FILE *f = fopen("fileName", "r");
readdata(f);
if(???)
{
puts("End of file was reached");
}

Which one of the following can replace the???? in the code above to determine if the end of a file has been reached?

- A. **feof(f)** B. eof(f) C. f == NULL D. !f

- 8 `f = fopen( filename, "r" );`
Referring to the code above, what is the proper definition for the variable `f`?
A. `FILE f;` B. `struct FILE f;` C. **`FILE *f;`** D. `int f;`
- 9 Which one of the following is valid for opening a file for only reading?
A. `fileOpen( "filenm", "r");` B. **`fopen( "filenm", "r");`**
C. `fileOpen( "filenm", "ra");` D. `fopen( "filenm", "read");`
- 10 What is a proper method of opening a file `test.bin` for writing as binary file?
A. `FILE *f = fwrite( "test.bin", "b" );`
B. `FILE *f = fopenb( "test.bin", "w" );`
C. **`FILE *f = fopen( "test.bin", "wb" );`**
D. `FILE *f = fopen( "test.bin", "write" );`

Short Questions

- 1 List file modes available to manage the file in C.
- 2 Explain the Append mode with example.
- 3 Write down the syntax to open the text file both in read and write mode.
- 4 Explain the `fclose()` function with example.
- 5 What do you mean by file pointer?
- 6 Which functions are used to read the character from the file and write character to the file?
- 7 Explain function(s) used to read the integer number from the file.
- 8 Differentiate: append mode and write mode
- 9 Differentiate: `printf` and `fprintf`
- 10 Differentiate: `scanf` and `fscanf`
- 11 Differentiate: `getc` and `getchar`
- 12 What is the function of `getc()` and `putc()`.
- 13 Explain `getw()` and `putw()` function in brief.
- 14 Explain `fprintf()` function with example.
- 15 Explain `fopen()` statement in brief.

Long Questions

- 1 Which I/O functions used in 'C'? What are the drawbacks of I/O operations? Which file handling functions are used in 'C'? Explain in brief.
- 2 How files can be defining in 'C'? Explain `fopen()` and `fclose()` function in detail.
- 3 Explain `getc()` and `putc()` function with example.

OR

Explain character handling functions used in file by giving example.

- 4 Explain fprintf() and fscanf() function with example.

OR

Which functions are used to read and write mixed data from/into a file? Explain in detail.

- 5 Explain getw() and putw() function with example.

OR

Explain functions used to handle integer numbers in file with example.



Shri. R. K. Parikh Arts & Science College, Petlad
B.Sc (Computer Science) – Semester – 4

Unit: 3 Introduction to Data Structures, Stack and Queue

❖ **Introduction to data structures, their usage, applications and advantages :-**

Meaning of Data Structures :-

- Representing information is fundamental to computer science. The primary purpose of most computer programs is NOT to perform calculations, but to store and retrieve information – usually as fast as possible. For this reason, the study of data structures and the algorithms that manipulate them is the heart of computer science.
- A data structure is used to organize the information in computer such a way that it satisfies our needs. A data structure is a specialized format for organizing and storing data.
- A data structure is any data representation and its associated operations. Even an integer or floating point number stored on the computer can be viewed as a simple data structure. More typically, a data structure is meant to be an organization or structuring for a collection of data items. A sorted list of integers stored in an array is an example of such a structuring.
- Any data structure is designed to organize data to suit a specific purpose so that it can be accessed and worked with in appropriate ways. In computer programming, a data structure may be selected or designed to store data for the purpose of working on it with various algorithms.
- A data structure is used to provide efficient processing for structuring information into memory.

Data Structure:

- Definition: **“It is a way of organizing data that not only the data items stored but also their relationship to each other”.**

Storage structure:

- Definition: The representation of a particular data in memory of a computer is called a storage structure.

File structure:

- Definition: A storage structure representation in auxiliary memory is called a file structure.

Applications of data structure:

The applications of data structure are:

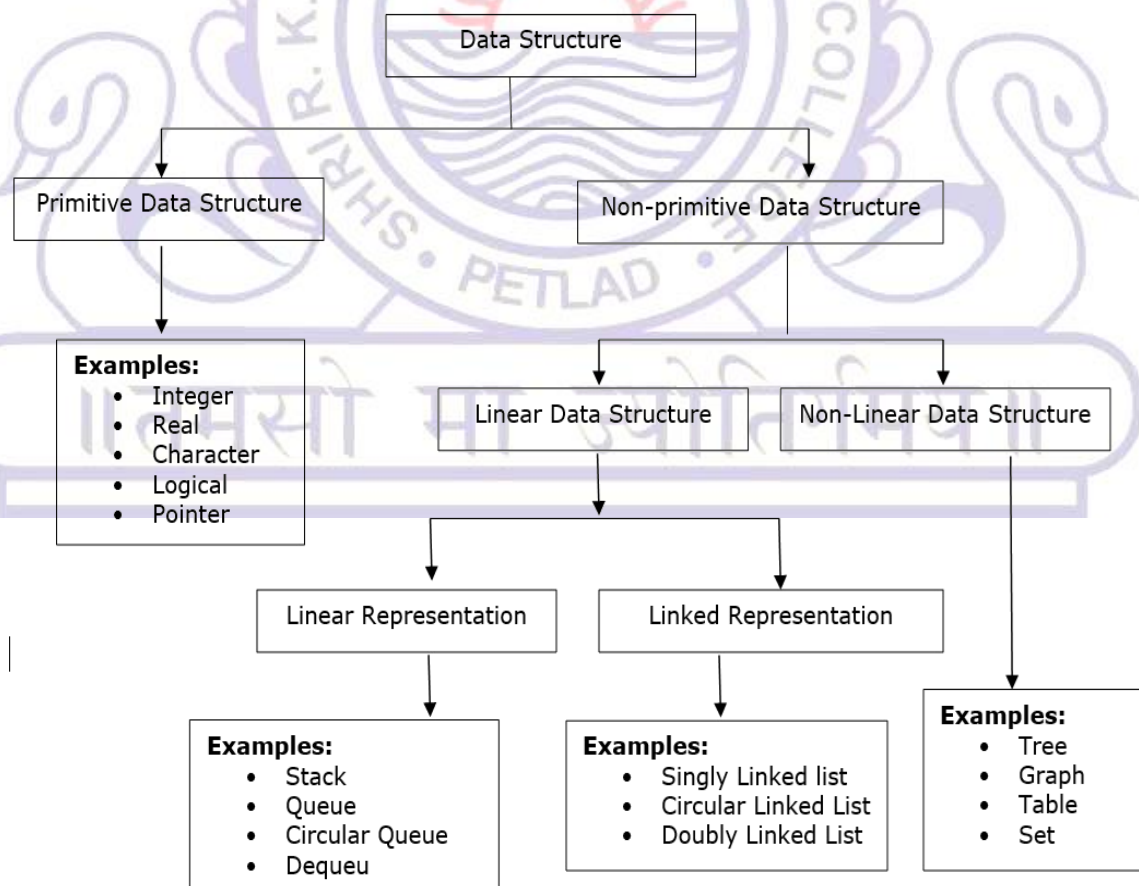
- Compiler design
- Operating system
- Database Management System
- Statistical analysis package
- Numerical analysis package
- Computer graphics
- Artificial Intelligence
- Simulation
- Network analysis

❖ Primitive and non-primitive data structures and operations on them, Linear and non-linear data structures :-

Classification of Data structure:

The data structure is classified into mainly two categories. They are

1. Primitive data structure
2. Non-primitive data structure



Data types:

1. Primitive data type
2. Non-primitive data type

	Primitive data type		Non-primitive data type
1.	A data type that is directly operated by machine level instruction is known as primitive data type.	1.	A data type that is structured set of primitive data types is known as Non-primitive data type.
2.	E.g. Integer, real, character, logical and pointer.	2.	E.g. Array, stack, queue, linked list and file.
3.	It is a fundamental data type.	3.	It is a set of primitive data type.
4.	There is no other name.	4.	It is also known as composite or complex data type.

Advantages of Data structure:

The major advantages of data structures are:

- It gives different level of organizing data.
- It tells how data can be stored and accessed in its elementary level

Operations on Data structure:

There are four basic operation carried out on primitive data structure. They are

1. CREATION
2. DESTROY
3. SELECTION
4. UPDATE

CREATION:

- An operation that is used to create a data structure is known as creation operation. A variable can be created in PASCAL, FORTRAN, ALGO, PL\I plus and many other languages by using a declaration statement.
- In C language, the following declaration statement
int n;
- creates a variable n. It also occupied the required memory to the variable n. In this case 2 bytes are occupied to the variable n.

DESTROY

- An operation that is provides the complementary effect of a CREATION operation is known as DESTROY operation.

- There are some languages such as FORTRAN does not allow to programmer to destroy a data structure, once they have been created. Some language such as C, C++, PASCAL, PL\I allow to programmer to destroy a data structure.
- In C language, the following statement
`free( ptr);` //It is previously declared as **int *ptr;** in a program.
- will destroy the variable 'ptr'. It means that to free the memory which is occupied to variable 'ptr'. Now, this variable is no longer use.

SELECTION

- An operation that is used to access data within the data structure is known as SELECTION operation. The form of selection operation is depends on the type of data structure being accessed.
- In C language, the following statement
`int a[5];`
 declares an array of five elements. Now, if we want to display an element from 4th position on the screen using following statement.
`printf("%d",a[3]);`
 It is a selection operation. This is because we access the elements within array data structure.

UPDATE

- An operation that is used to change data in the data structure is known as UPDATE operation.
- E.g. An assignment statement (=) is a good example of an update operation. In C++, the following statement
`jawab = a + b;`
 will total the value of variables a and b, and stores the result into jawab variable.

Categories of data structure Data structure:

A data structure is classified into two categories:

1. Linear data structures
2. Non-Linear data structures. .

Linear data structures:

- **A data structure is said to be linear if all the elements are traversed sequentially.**
- All the elements of a linear data structure will form a sequence.
- A Linear data structure maintains a linear structure. Therefore, we can traverse either forward or backward from a node.
- There are two ways of representing linear data structures in memory.
- One way is to have the linear relationship between the elements by means of **sequential** memory locations.
- Such linear structures are stack, queue and array.
- The other way is to have the linear relationship between the elements represented by means of **links**.
- Such linear data structure is linked list.

➤ **Examples** of Linear data structure are:

- Array
- stack
- queue
- Linked list

Non-Linear data structures:

- A data structure is said to be non-linear if all the elements are NOT traversed sequentially.
- All the elements of a non-linear data structure will not form a sequence.
- A non-Linear data structure does not maintain a linear structure. It branches to more than one node and cannot be traversed in a single run.
- **Examples** of non-Linear data structure are:
 - Trees
 - Graphs
 - Tables
 - Sets

❖ **Stack : Introduction to stacks, operations on stacks, Applications of stacks:-**

Stack

Definition: It is a data structure in which insertion and deletion of elements occur only at one end.

Characteristics:

A stack data structure has following characteristics:

- It is non-primitive data structure.
- Its nature is LIFO (Last In First Out).
- The insertion and deletion operation occur only at one end.
- It is linear data structure of variable size.
- The elements are removed from the opposite order from that in which they are added to the stack.

Operations on Stack:

They are:

- Push
- Pop
- Peep
- Change

Push:

- An insertion operation is known as Push.
- To insert an element on a stack.
-

Pop:

- A deletion operation is known as Pop.
- To delete an element from a stack.

Peep:

- To give the value of i^{th} element from a stack.

Change:

- To change the value of i^{th} element from a stack.

Top and Bottom of a stack :

- The most accessible element of a stack is known as Top of a stack.
- The least accessible element of a stack is known as Bottom of a stack.

Representation of a stack :

A stack can be represented by a vector as follow:



Examples:

Ordering pages in a file, Marbles in a test tube.

Algorithm :

[1] An algorithm to insert an element into a stack.

PUSH (S, TOP, N, X) : This algorithm is to insert an element X into a stack S which is consisting of N elements. Top indicates the position of the top element from the stack.

Step -1 : [Check for Overflow]

If (TOP $\geq$ N)

Then

Write ("STACK OVERFLOW")

Return

Step -2 : [Increment Top]

TOP $\leftarrow$ TOP + 1

Step -3 : [Insert an Element]

S[TOP] $\leftarrow$ X

Step -4 : [Finished]

Return

[2] An algorithm to delete an element from a stack.

POP (S, TOP) : This algorithm is to delete an element from a stack S which is consisting of N elements. Top indicates the position of the top element from the stack.

Step -1 : [Check for Underflow]

If (TOP = 0)

Then

Write ("STACK UNDERFLOW")

Return

Step -2 : [Decrement Top]
 $TOP \leftarrow TOP - 1$

Step -3 : [Return an Element]
 Return (S[TOP + 1])

[3] An algorithm to give the value of i^{th} element from a stack.

PEEP (S, TOP, N, I) : This algorithm is to give the value of i^{th} element from a stack S which is consisting of N elements. Top indicates the position of the top element from the stack.

Step -1 : [Check for Underflow]
 If ((TOP - I + 1) <= 0)
 Then
 Write ('STACK UNDERFLOW' ON PEEP)
 Return

Step -2 : [Return an Element]
 Return (S[TOP - I + 1])

[4] An algorithm to change the value of i^{th} element from a stack.

CHANGE (S, TOP, N, I, X) : This algorithm is to change the value of i^{th} element to a new value X. Top indicates the position of the top element from the stack.

Step -1 : [Check for Underflow]
 If ((TOP - I + 1) <= 0)
 Then
 Write ('STACK UNDERFLOW' ON CHANGE)
 Return

Step -2 : [Change an Element]
 $S[TOP - I + 1] \leftarrow X$

Step -3 : [Finished]
 Return

Applications of Stack :

1. Recursion
2. Polish Expression and their compilation
3. Stack Machine

❖ Queues : Queues and their uses, Types of queues : Simple queues, Circular queues, Double ended queues :-

Queue:-

Definition: It is a data structure in which insertion of an element occurs at one end and deletion occur at other end.

OR

Queue is a data structure in which elements are added to one end (rear) and removed from the other end (front) in FIFO (First In First Out) structure.

Characteristics of Queue:

A queue data structure has following characteristics:

- Its nature is FIFO (First In First Out).
- The insertion and deletion operation occur at opposite end.

Representation of a Queue:

A queue can be represented by a vector as follow:



Examples:

- People standing at Ticket Window
- Vehicles at a Traffic Signal

Types of a Queue:

1. Simple Queue OR Queue
2. Circular Queue
3. Dequeue (Double Ended Queue)
4. Priority Queue

Simple Queue :

- A Queue has two pointer, front pointer and rear pointer, which are pointing to front and rear element of the queue.
- Initially value of both pointer is zero(0). The value of front pointer is zero, indicates queue is empty. The value of rear pointer N indicates queue is full. Where N is total number of element in queue(size of queue).
- If front pointer's value is zero indicate queue is empty, so deletion is not possible. And if rear pointer's value is N then insertion is not possible.
- When you Insert an element value of rear pointer is increased by 1. When you delete an element value of front pointer is increased by 1.
- If front pointer and rear pointer both are equal and if we remove(delete) an element then after deletion value of front and rear pointer is set to be zero.

Algorithm:

[1]. An algorithm to insert an element at the rear end of a Queue.

QINSERT (Q, F, R, N, X) : This algorithm is to insert an element X at the rear end of a Queue which is consisting of N elements. F and R are the position of the front and rear element from the queue respectively.

Step -1 : [Check for Overflow]

 If (R >= N)

 Then

 Write ('QUEUE OVERFLOW/FULL')

 Return

Step -2 : [Increment rear pointer]

$R \longleftarrow R + 1$

Step -3 : [Insert an Element]

$Q[R] \longleftarrow X$

Step -4 : [Set Front pointer]

If ($F = 0$)

Then

$F \longleftarrow 1$

[2]. An algorithm to delete an element from the front end of a Queue.

QDELETE (Q, F, R, N) : This algorithm is to delete an element from the front end of a Queue which is consisting of N elements. F and R are the position of the front and rear element from the queue respectively.

Step -1 : [Check for Underflow]

If ($F = 0$)

Then

Write ("QUEUE UNDERFLOW/EMPTY")

Return

Step -2 : [Delete Element]

$X \longleftarrow Q[F]$

Step -3 : [Check for Queue Empty]

If ($F = R$)

Then

$F \longleftarrow R \longleftarrow 0$

Else

$F \longleftarrow F + 1$

Step -4 : [Return an element]

Return (X)

Application of Queue:

A well known application of a queue is **Simulation**

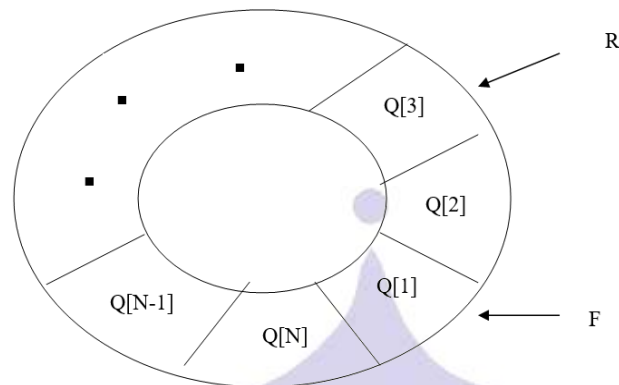
Circular Queue:-

- There is a major drawback using simple queue because we know that in simple queue, all the insertion is done on rear pointer end, & all the deletion are possible on front end. The problem with simple queue is that once we have inserted the elements and after removing elements using front pointer these elements becomes blank, because all the insertions will take place on rear end. So the positions which are removed by front pointer are always blank.
- In simple queue if $\text{rear} = \text{size of queue}$, no further insertion is possible though there is a free space in leading elements. The solution for this problem is **Circular Queue**.
- If we have an array CQ, that contain N element in which $\text{CQ}[1]$ comes after $\text{CQ}[N]$ in the array, when this technique is used to construct a queue then queue is called **Circular Queue**.
- **Definition:** A queue in which the elements are arranged in the circular fashion is known as Circular queue.

OR

A queue is called Circular Queue when last element comes just before first element.

Representation of a Circular Queue:



- In Circular queue, when rear pointer = N(Size), and if we insert an element then this element is assigned to CQ[1], instead of increasing rear pointer to rear+1, we reset rear pointer to 1.
- Similarly if front pointer value reach at maximum(N), front = N and we want to remove element from cqueue, then we reset a front pointer to 1 instead of n+1.
- Suppose a cqueue contain only one element, that is (front=rear) != 0, and if we remove that element, then front and rear pointer is assign a value zero(0) to indicate cqueue is empty.

INSERTING ANY ELEMENT IN CIRCULAR QUEUE

Step 1: Check for a Overflow

If $(F = 1 \text{ and } R = N) \text{ or } F=R+1$ Then

Display "Circular Queue Overflow/Full"

return

Step 2: Set the Rear Pointer

If $R = N$ Then

$R \leftarrow 1$

Else

$R \leftarrow R + 1$

Step 3: Insert the Element

$CQ[R] \leftarrow x$

Step 4: Set the Front Pointer

If $(F = 0)$ Then

$F \leftarrow 1$

Step 5: Exit

DELETION OF ELEMENT FROM CIRCULAR QUEUE

Step 1: Check for an Underflow

If $F \leq 0$ Then

Write "Circular Queue is empty/underflow "

return

Step 2: Store the Element that is to be deleted

$x \leftarrow CQ[F]$

Step 3: Set the value of Front and Rear Pointer

If $F = R$ Then

$F \leftarrow 0, R \leftarrow 0$

Step 4: Increment the Front Pointer

If $F = N$ Then

$F \leftarrow 1$

Else

$F \leftarrow F + 1$

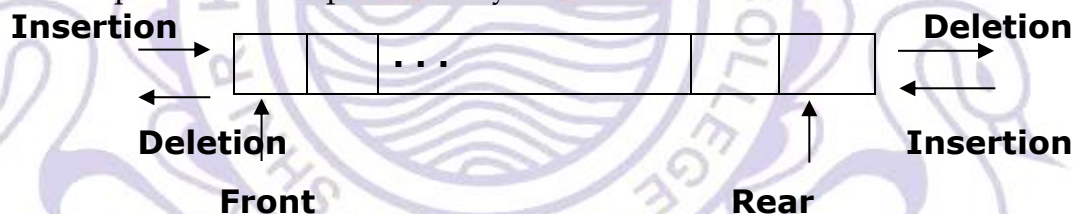
Step 5: Return the Element that is to be deleted

return x

Step 6: Exit

Double Ended Queue:

- A queue in which insertion as well as deletion occurs at both the end is known as double ended queue.
- It is also known as Dequeue.
- In stack insertions and deletions are possible only at one end (LIFO). While in queue insertions at one end and deletion at other end (FIFO). Thus we can say that dequeue is more superior representation of linear list as compared to stack and Simple Queue.
- A double ended queue can be represented by a vector as follow:



There are two types of Deque:

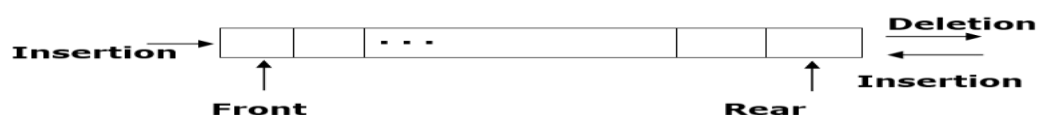
1. Input-Restricted Deque :

In input restricted Dequeue it allow insertion at only one end of the list but allows deletion at both end of the list.



2. Output-Restricted Deque

Deletion at one end, but insertion are possible on both end is called Output Restricted Dequeue.



PRIORITY QUEUES :

- A queue in which we are able to insert or remove items from any positions based on some priority is called priority queue.

	R1	R2	R3		Ri	O1	O2		Oj
Priority →	1	1	1		1	2	2		2

R1	R2	R3		Ri	← Priority - 1
----	----	----	-------	----	----------------

O1	O2	O3		Oj	← Priority-2
----	----	----	-------	----	--------------

- A priority queue can be conceptualized as a series of queues representing situations in which it is known a priori what priorities are associated with queue items.
- Above figure shows how the single priority queue can be visualized as two separate queues, each exhibiting a strictly FIFO behavior. Elements in the second queue can be removed only when the first queue is empty.
- The separation of single priority queue into a series of queues also suggests an efficient storage representation of a priority queue. When elements are inserted, they are always added at the end of one of the queues as determined by the priority.
- Alternatively, if a single sequential storage structure is used for the priority queue then insertion may mean that the new element must be placed in the middle of the structure. This can require the movement of several items. It is better to split the priority queue into several queues, each having its own storage structure.

Difference between Stack and Queue :

Stack	Queue
1. Stack is a special type of list in which insertion and deletion are made at the one end.	1. Queue is a list in which insertion occurs at one end and deletion occurs at other end
2. It is list which support LIFO nature.	2. It is list which support FIFO nature.
3. Insertion operation is known as PUSH.	3. End of list is known as REAR, where insertion can be done.
4. Deletion operation is known as POP.	4. Beginning of the list is known as FRONT where deletion is done.

Shri. R. K. Parikh Arts & Science College Petlad

US04CCSC51- Advanced C Programming and Introduction to Data Structures

QUESTION BANK

UNIT -3 (MCQ)

- 1 Files are a _____ type of Data Structure.
A. Linear B. Primitive **C. Non-Primitive** D. Non-Linear
- 2 Which of the following data structures are indexed structures?
A. linear arrays B. Linked List C. Both A and B D. None of these
- 3 Two dimensional arrays are also called?
A. tables arrays B. Matrix arrays **C. Both A and B** D. None of these
- 4 Which of the following data structure store the homogeneous data elements?
A. Arrays B. Pointers C. Records D. None of these
- 5 When new data are to be inserted into a data structure, but there is no available space; this situation is usually called?
A. underflow **B. overflow** C. housefull D. saturated
- 6 The term "push" and "pop" is related to the?
A. Array B. Stacks **C. Queue** D. All of the above
- 7 A data structure where elements can be added or removed at either end but not in the middle?
A. Linked List B. Queue C. Stack **D. Dequeue**
- 8 A stack is _____ type of data structure.
A. Linear B. Non-linear C. Both A and B D. None of these
- 9 Which of the following is an operation of a Stack data structure?
A. Top B. Bottom **C. Peep** D. None of these
- 10 Which of the following statement is FALSE for the Stack data structure?
A. Its nature is LIFO **B. Its nature is FIFO**
C. It is non-primitive data structure D. It is a linear data structure

- 11 A data structure in which insertion and deletion of an elements occurs at only one end is known as _____.
A. Queue **B. Stack** C. Tree D. Graph
- 12 Which of the following is an operation of a Stack data structure?
A. Push B. Peep C. Pop **D. All of the above**
- 13 An operation that is used to insert an element on a stack is known as _____.
A. Push B. Pop C. Peep D. Change
- 14 An operation that is used to delete an element from a stack is known as _____.
A. Push **B. Pop** C. Peep D. Change
- 15 An operation that is used to give the value of an element at a particular position from a top of a stack is known as _____.
A. Push B. Pop **C. Peep** D. Change
- 16 An operation that is used to change the value of an element at a particular position from a top of a stack is known as _____.
A. Push B. Pop C. Peep **D. Change**
- 17 Which of the following is TRUE for a Stack data structure?
A. Linear B. Non-primitive **C. Both A and B** D. None of these
- 18 Which of the following is NOT an application of a Stack data structure?
A. Stack Machine B. Evaluation of an Expression
C. Recursion **D. Simulation**
- 19 Which of the following statement is FALSE for the Queue data structure?
A. Its nature is LIFO **B. Its nature is FIFO**
C. It is non-primitive data structure D. It is a linear data structure
- 20 A data structure in which insertion of an element occurs at one end and deletion of an element occurs at other end is known as _____.
A. Tree B. Graph **C. Queue** D. Stack
- 21 A data structure in which insertion and deletion of an elements occurs at both the end is known as _____.
A. Stack B. Queue C. Priority Queue **D. Deque**
- 22 Which of the following is TRUE for a Queue data structure?
A. Linear B. Non-primitive **C. Both A and B** D. None of these

Short Questions

- 1 Define: Data Structure, Stack
- 2 Define and explain in brief: Primitive Data Structure
- 3 What is non-primitive Data Structure?
- 4 Give the Example of Primitive Data Structure.
- 5 Give the Example of Non-Primitive Data Structure.
- 6 What do you mean Linear Data Structure?
- 7 What do you mean by Non-Linear Data Structure?
- 8 Draw the Hierarchical Structure of Data Structure.
- 9 List out different applications of data Structure.
- 10 Which are the main operations that can be performed on Data Structure?
- 11 Differentiate: primitive and non primitive data structure
- 12 What is a Stack? Give examples.
- 13 State various Applications of Stack.
- 14 List operations on a Stack. Define ANY ONE of them.
- 15 Give representation of a Stack data structure
- 16 What do you mean by Top and Bottom of a Stack?
- 17 What is a Queue? Give examples.
- 18 State various types of queue.
- 19 Define: Queue and Deque
- 20 Give representation of a Queue data structure.
- 21 Define : Circular Queue and Priority Queue.

Long Questions

- 1 Explain data structure with examples. Also write advantages of data structure.
- 2 Write a short note on primitive data structure operations.
- 3 Write a short note on linear data structure.
- 4 Write a short note on non linear data structure.
- 5 Explain the linear and non linear data structure briefly.
- 6 Write an algorithm to insert an element into a Stack.
- 7 Write an algorithm to delete an element from a Stack.
- 8 Write an algorithm for Peep operation of a Stack.
- 9 Write an algorithm for Change operation of a Stack.
- 10 Write an algorithm to insert an element into a simple queue.
- 11 Write an algorithm to delete an element from a simple queue.

Very Long Questions

- 1 Explain a STACK with an example. Write algorithm to perform various operations over a stack.
- 2 Explain a Queue with an example. Write algorithm to perform various operations on a queue.



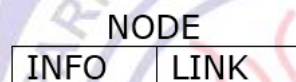
Shri. R. K. Parikh Arts & Science College, Petlad
B.Sc (Computer Science) – Semester – 4

Unit: 4 Linked Lists, Sorting and Searching Techniques

❖ **Introduction to linked lists : Types of linked lists , Singly linked lists, doubly linked lists, Circular linked lists, Applications of linked lists**

Introduction to linked lists :-

- A data structure that contains a data field as well as pointer field is known as linked list.
- A node structure is as follow:



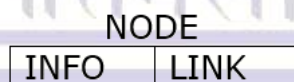
- A node of the linked list consist two fields:
 - An information field is represented by INFO
 - A pointer field is represented by LINK

Types of Linked List :-

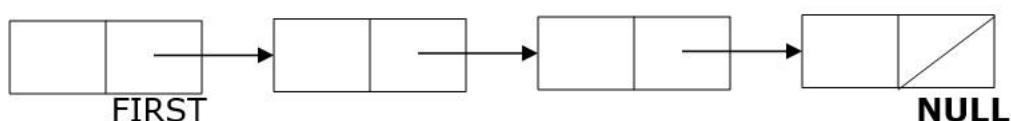
1. Singly linked list
2. Circular Linked list
3. Doubly linked list

Singly Linked List :-

- A node structure of singly linked list is as follow.



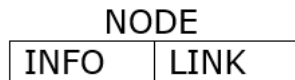
- A node of the linked list consist two fields:
 - An information field is represented by INFO
 - A pointer field is represented by LINK. **It is pointing to the next node of a list. A linked list in which pointer fields of each node is pointing to the next node is known as singly linked list.**



- In Singly linked list,
 - An address of the first node is represented by FIRST.
 - A right pointer field of last node is set to NULL

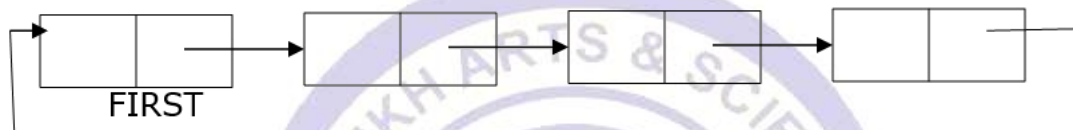
Circular Linked List :-

- A node structure of Circular linked list is as follow.



- A node of the linked list consist two fields:
 - An information field is represented by INFO
 - A pointer field is represented by LINK.

A Singly linked list in which pointer fields of last node is pointing to the first node is known as circular linked list.



- In Circular linked list,
 - An address of the first node is represented by **FIRST**.
 - A right pointer field of last node is set to address of FIRST node.

Doubly Linked List :-

A node structure of doubly linked list is a follow.

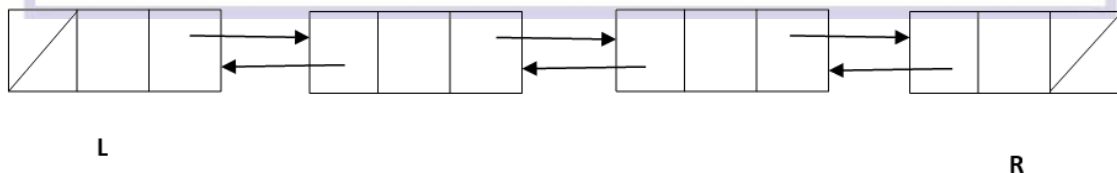


LPTR – pointing to the left node

RPTR – pointing to the right node

INFO – actual value

A data structure that has an information field and two pointers fields is known as doubly linked list. Each of the pointer field contains the address of left or right node. An information field contains the actual data about node.



- In doubly linked list,
 - An address of the first node is represented by L.
 - An address of the last node is represented by R.
 - A left pointer field of first node and right pointer field of last node is set to NULL.

ALGORITHMS:

An algorithm to insert a new node into singly linked list at beginning position.

INSERT (FIRST, X) : This algorithm is used to insert a new node into singly linked list at beginning position. An address of the first node is given by FIRST.

- Step 1: Create a new node
Newnode $\leftarrow$ getnode()
- Step 2: Assign value to the node
Info(newnode) $\leftarrow$ x
Next(newnode) $\leftarrow$ First
- Step 3: First $\leftarrow$ newnode
- Step 4: Return the first pointer
Return(First)
- Step 5: Exit

An algorithm to insert a new node into singly linked list at ending position.

INSERT (FIRST, X) : This algorithm is used to insert a new node into singly linked list at ending position. An address of the first node is given by FIRST.

- Step 1: Create a new node
Newnode $\leftarrow$ getnode()
- Step 2: Assign value to the new node
Info(newnode) $\leftarrow$ x
Next(newnode) $\leftarrow$ null
- Step 3: Set value of First Pointer
If first = null then
 First $\leftarrow$ newnode
 return(First)
End if
- Step 4: Move through the list
Temp=first
Repeat While(next(temp) $\neq$ null)
 Temp=next(temp)
Next(temp) $\leftarrow$ newnode
- Step 5: Return the First Pointer
return(First)
- Step 6: Exit

An algorithm to insert a new node into singly linked list such that it maintains the ordering of the information field in a ascending order.

INSERT (FIRST, X) : This algorithm is used to insert a new node into singly linked list such that it maintains the ordering of the information field in a ascending order. An address of the first node is given by FIRST.

- Step 1: Create the node
Newnode $\leftarrow$ getnode()
- Step 2: Assign the value to the new node
Info(newnode) $\leftarrow$ x
- Step 3: If First = null Then new node will be inserted into beginning
First $\leftarrow$ newnode
Return First
End if
- Step 4: Check for the smallest node
If info(newnode) $\leq$ info(first) Then
Next(newnode) $\leftarrow$ first
First $\leftarrow$ newnode
End if
- Step 5: Search for the node having greater value than Info(newnode)
Temp=first
While((info(temp) $\leq$ info(newnode)) and (temp $\neq$ null))
Prev $\leftarrow$ temp
Temp $\leftarrow$ next(temp)
End while
Next(newnode) $\leftarrow$ temp
Next(prev) $\leftarrow$ newnode
- Step 6: Return the First Pointer
return(first)
- Step 7:Exit

An algorithm to delete a first node from a singly linked list.

INSERT (FIRST, X) : This algorithm is used to delete a first node from a singly linked list.

- Step 1: Check whether the list is empty or not.
If first = null Then
Write ("List is empty")
End if
- Step 2: Copy and update the first pointer

Temp ← first
First ← next(Temp)
Next(temp) ← null

Step 3: Delete the node from the linked list
Freenode(temp)

Step 4: Return the first pointer
return (first)

Step 5: Exit

Application of Linked List :-

- The following are the application of Linked list:
 1. Polynomial manipulation
 2. Linked dictionary
 3. Multiple-Precision arithmetic

❖ Sorting and Searching Techniques : Basic sorting techniques (Bubble, Selection, Insertion), Searching techniques (Sequential and Binary)

Sorting :-

- Sorting means arranging data in specified order either Ascending or Descending.
- Sorting : Sorting techniques can be divided in two categories as
 1. Internal sorting techniques
 2. External sorting techniques.
- In Internal sorting technique, data that is going to be sorted will be in main or primary memory. While in External sorting technique, data will be on auxiliary memory like floppy disc or on tape.
- Here we learn only internal sorting technique. There are many methods for sorting, which is the best method is very difficult to say because it is depending on the situation and complexity. Some methods perform better in some situation, while other cannot perform well. So we should know the efficiency of any sorting technique.

Sorting Methods :- Following are sorting techniques.

1. Bubble sort
2. Selection sort
3. Quick sort
4. Insertion sort
5. Merge sort
6. Heap sort
7. Address calculation sort
8. Radix sort

From above sorting techniques we learn Bubble sort, Selection sort and Insertion sort only.

Bubble Sort :-

- This sorting method is based on comparing and exchanging pairs of element in array. One round of scanning through the set of element is called passing elements.
- The bubble sort method drives its name from the fact that the smallest data item bubbles up to the top of an array (In ascending order case).
- The algorithm begins by comparing first and second element. If first element greater than second element then interchange elements otherwise keep as it is. Compare Second and third element and do same procedure and repeat this comparison up to the last pair and you complete first pass. After end of first pass largest element will be placed at the last position.
- For complete sorting you need to require $n-1$ passes and finally you get the smallest element at the top of array.

Data : 55 44 22 33 11

Pass - 1

55	44	44	44	44
44	55	22	22	22
22	22	55	33	33
33	33	33	55	11
11	11	11	11	55

Pass - 2

44	22	22	22
22	44	33	33
33	33	44	11
11	11	11	44
55	55	55	55

Pass - 3

22	22	22
33	33	11
11	11	33
44	44	44
55	55	55

Pass-4

22	11
11	22
33	33
44	44
55	55

Algorithm : BUBBLE SORT

Bubble(data, n) --> Data is unsorted array and n total number of element.

- Step - 1 Repeat for i = 1 to n
 repeat step - 2.
- Step - 2 Repeat for j = 1 to n-i
 if data[j]>data[j+1]
 swap data[j],data[j+1]
- Step - 3 Display Sorted data
- Step -4 Exit

Complexity of Bubble Sort :-

- In Bubble sort total comparison in first pass is n-1. in second pass it is n-2 and so on. The best case of bubble sort is in which only one iteration is required.
- In that number of iteration required is n-1. the worst case is when the given array is arranged in reverse order. So in this case total iteration required is $n(n-1)/2$ which is of $O(n^2)$.
- In average case one must find the expected of iteration first, in this case extra iteration required to check that the sorting is completed.

Advantages :

- Simple algorithm
- Easy to implement

Selection Sort :-

- Selection sort is the easiest way to sort array elements. A table beginning with the first element or record, a search is performed on it to locate the smallest key.
- When this element is found, it is interchange with the first element. This interchange places the element with the smallest key in the first position of an array.
- A search for the second smallest key is carried on. i.e. it examine element/record from the second element/record onwards.
- The element which has the smallest key is interchanged with the element located at the second position of the array.
- The process for searching for the record with the next smallest key and placing on its proper position continues until all elements have been sorted.

Let us take an example to understand this process.

Pass

K=1 & LOC=4	77	33	44	11	88	22	66	55
K=2 & LOC=6	11	33	44	77	88	22	66	55

K=3 & LOC=6	11	22	44	77	88	33	66	55
K=4 & LOC=6	11	22	33	77	88	44	66	55
K=5 & LOC=8	11	22	33	44	88	77	66	55
K=6 & LOC=7	11	22	33	44	55	77	66	88
K=7 & LOC=1	11	22	33	44	55	66	77	88
Sorted Order	11	22	33	44	55	66	77	88

Algorithm : Selection Sort

Selection(A, N) ----> A is an array and N total number of element

Step - 1 [Loop for controlling passes]
Repeat for K=1 to N-1
repeat Step - 2 to Step - 4
go to Step - 5

Step - 2 [Assign sml and LOC]
set sml = A[K] and LOC = K

Step - 3 [Inner loop to find minimum value]
Repeat for J = K+1 to N
If sml > A[J] then
 sml = A[J]
 LOC = J
endif

Step - 4 [Interchange minimum value]
Interchange A[K] and A[LOC]

Step - 5 Display sorted array

Step - 6 Exit

Complexity of Selection sort :-

- Selection sort algorithm search smallest elements in table and then put the element at proper position. In pass 1 it will compare n-1 element, in pass 2, n-2 element are compared and so on. So the complexity in
Average case is $O(n^2)$
Worst case is $O(n^2)$
Best case is $O(n^2)$

- Selection sort does not see the order of the element in the table that is why complexity is same for all cases.

Advantages:-

1. In every pass an element will be at its proper position.
2. Few temporary variables required.
3. Simple algorithm.

Insertion Sort :-

- In Insertion sort, we have to present a particular element at appropriate position. In first iteration 2nd element is compared with 1st element, in second iteration 3rd element is compared with 1st and 2nd element and so on.
- If at any point it found that element can be presented at a position then space is created for it by shifting the other element one position to the right and element is inserted at suitable position. This method is same as playing cards where we place the cards in proper order. Let us take an example to understand this process.

Data :	77	33	44	11	88	22	66	55
Pass								
K=1	77	33	44	11	88	22	66	55
K=2	77	33	44	11	88	22	66	55
K=3	33	77	44	11	88	22	66	55
K=4	33	44	77	11	88	22	66	55
K=5	11	33	44	77	88	22	66	55
K=6	11	33	44	77	88	22	66	55
K=7	11	22	33	44	77	88	66	55
K=8	11	22	33	44	66	77	88	55
Sorted	11	22	33	44	55	66	77	88
Order								

Algorithm : Insertion Sort

Insertion(A, N) ----> A is an array and N total number of element

Step - 1 Repeat Step-2 to Step-4 for k = 2 to N
go to Step - 5.

Step - 2 Set temp=A[k] and ptr= k - 1

Step - 3 Repeat while(temp<A[ptr] & ptr>0)
(a) set A[ptr+1]=A[ptr] (moves element forward)

(b) $\text{ptr} = \text{ptr} - 1$

Step - 4 Set $A[\text{ptr}+1]=\text{temp}$ (Insert element in proper position)

Step - 5 Return

Complexity of Insertion sort :-

- In Insertion sort an element before or after the element, so in first insertion no comparison will be made, In third iteration 2 comparisons will be made and so on. So in worst case where all elements are in reverse order the complexity is $O(n^2)$.
- If the table is sorted then only one comparison is made in each iteration. So the sort is $O(n)$. Speed of sort can be improved by applying binary search to find the position of the element.

Advantage :-

1. Simplicity
2. Efficient where number of elements to be sorted are very less.
3. It processes very fast where table is near about sorted.

Searching :-

- Searching is to find a particular element from the table of data. The search will give either element found or element not found. And if element found then the search is successful otherwise unsuccessful.
- There are two main search methods.
 1. Linear or Sequential search
 2. Binary search

Linear Search :-

- Linear search is a very simple method. It searches the element in a sequence in the list. This method can be implemented on sorted list or unsorted list. If the table of data is unsorted then searching starts from the first element and continues until the element is found or the end of the table is reached. Let us take an example to understand this search method.
59 6 14 10 8 11 12
- The above table of data is having unsorted elements. If we want to find 10 then we start comparing 10 from the first element, if it equals we stop otherwise continue until data is found or the table ends.
- The performance of the linear search can be measured by counting the comparisons done to find out an element, in this search technique the number of search comparisons required is $O(n)$.

Algorithm : Linear Search

Linear_Search(A, N, ele) ---> A is an array, N total element and ele is an element to be searched.

Step - 1 Initialize $\text{flag} = 0$

Step - 2 Repeat Step - 3 for k = 1 to N
 Step - 3 if (A[k] = ele) then
 flag = 1
 break
 Step - 4 If (flag = 1)
 display " Search is successful"
 else
 display " Search is unsuccessful"

Step - 5 Return

Binary Search :-

- This method of searching is very difficult. The prerequisite of binary search, is data from which we want to search must be in sorted order. In this method to search an element we compare it with the middle element of the list, if it matches with the element which we want to search then search will be successful and we stop otherwise the list is divided into two halves, one from the first element to the middle element and other from the middle element to the last element. As a result all the elements in the first half are smaller than middle element while all the elements in the second half are greater than the center elements.
- The searching will process either of the two halves depending upon the target element is greater or smaller than the center element. If the element is smaller than the middle element that process will be for the first half otherwise for the second half.
- In this search, three things are performed low, high and middle, where low represents lower limit of the list which is always 1, high represents upper limit of the list which is always n and middle which is the average of low and high.
 i.e. $mid = (low + high) / 2$
 We compare middle element with key.
 If (Ele < A[mid]) then
 high = mid - 1
 else If (Ele > A[mid]) then
 low = mid + 1
 else If (Ele == A[mid]) then
 Display proper message

Repeat above step until low <= high. i.e. This process continues until the entire list is traversed or key is searched. At last if not found, display appropriate message.

Algorithm of Binary Search Method :

Binary_Search(A,N,ele) ---> A is an array, N total element & ele is element to be searched

Step 1 : [Find middle element]
 low=1
 high=n
 Mid=(low + high) / 2

Step 2 : Repeat step 3 while $ele \neq a[mid]$ && $low \leq high$

Step 3 : [Compare element]

If ($ele < a[mid]$)

high= $mid-1$

Else

If($ele > a[mid]$)

low= $mid+1$

Step 4 : [Search is found]

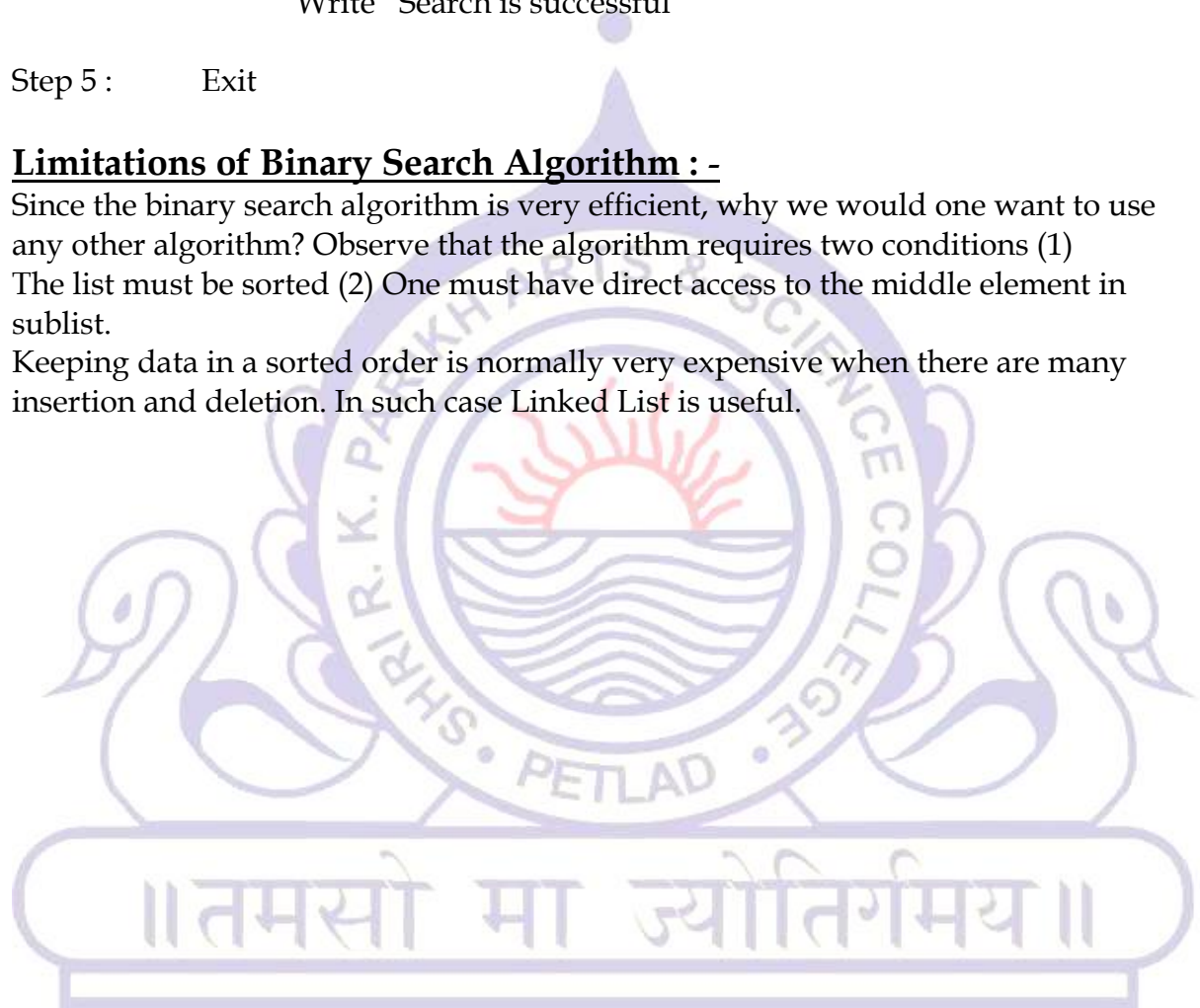
If($ele = a[mid]$) Then

Write "Search is successful"

Step 5 : Exit

Limitations of Binary Search Algorithm : -

- Since the binary search algorithm is very efficient, why we would one want to use any other algorithm? Observe that the algorithm requires two conditions (1)
- The list must be sorted (2) One must have direct access to the middle element in sublist.
- Keeping data in a sorted order is normally very expensive when there are many insertion and deletion. In such case Linked List is useful.



Shri. R. K. Parikh Arts & Science College Petlad

US04CCSC51- Advanced C Programming and Introduction to Data Structures

QUESTION BANK

UNIT - 4 (MCQ)

- 1 A linked list is ____ type of data structure?
A. Linear B. Non-Linear C. Both (A) and (B) D. None of Above
- 2 Which of the following is NOT the type of Singly linked list?
A. Two-way list B. Doubly Linked list
C. Three-way list D. Circular Linked list
- 3 A data structure that contains not only a data field but also contains pointer field is known as _____.
A. Stack **B. Linked List** C. Queue D. Tree
- 4 A linked list in which last node pointing to the first node is known as _____.
A. Singly linked list B. Doubly linked list
C. Circular linked list D. None of these
- 5 A linear collection of data elements where the linear node is given by means of pointer is called?
A. Linked List B. Node List C. Primitive List D. None of these
- 6 In linked list each node contain minimum of two fields. One field is data field to store the data second field is?
A. Pointer to character B. Pointer to integer
C. Pointer to node D. Node
- 7 In doubly linked lists, traversal can be performed?
A. Only in forward direction B. Only in reverse direction
C. In both directions D. None
- 8 A variant of the linked list in which none of the node contains NULL pointer is?
A. Singly linked list B. Doubly linked list
C. Circular linked list D. None of these
- 9 Linked list is considered as an example of _____ type of memory allocation.
A. Dynamic B. Static C. Compile time D. None of these
- 10 In Linked List implementation, a node carries information regarding _____.
A. Data **B. Link** C. Data & Link D. None of these

- 11 If the given input array is sorted or nearly sorted, which of the following algorithm gives the best performance?
A. Insertion Sort B. Selection Sort C. Quick Sort D. None of these
- 12 Which of the following algorithm pays the least attention to the ordering of the elements in the input list?
A. Insertion Sort **B. Selection Sort** C. Quick Sort D. None of these
- 13 Time complexity of bubble sort in best case is _____.
A. $\theta(n)$ B. $\theta(n \log n)$ C. $\theta(n^2)$ D. $\theta(n(\log n)^2)$
- 14 Finding an element, whether that is present in a list or not. It is called _____.
A. Sorting **B. Searching** C. Storing D. None of these
- 15 Binary search is ____ type of search.
A. Linear search B. Non linear search
C. Address calculation search D. Both A and B

Short Questions

- 1 What is a Linked List? How is it represented?
- 2 State various Applications of Linked List.
- 3 Describes the different types of Linked List.
- 4 What is a Singly Linked list?
- 5 What is a Circular Linked list?
- 6 What is a doubly Linked list?
- 7 Differentiate between singly linked list and doubly linked list.
- 8 List sorting techniques used to sort elements of a list.
- 9 List searching techniques used to search element from list. Explain any one in brief.
- 10 Write complexity of bubble sort.
- 11 What are the limitation of binary search?
- 12 Write an algorithm to search an element using sequential search.
- 13 Explain insertion sort in brief.
- 14 Explain selection sort in brief.
- 15 Explain bubble sort in brief.

Long Questions

- 1 Write an algorithm to insert an element at the beginning of a Singly linked list.
- 2 Write an algorithm to delete an element from a Singly linked list.
- 3 Write an algorithm to insert an element at the ending of a Singly linked list.
- 4 Write an algorithm to insert an element into a Singly linked list that maintains ascending order of elements.

- 5 What is linked list? Write a short note on Singly linked List.
- 6 Which sorting techniques are used to sort an array? Explain any one with example.
- 7 Which searching techniques are used to search element from array? Explain any one with example.
- 8 Explain bubble sort technique? Write an algorithm for bubble sort.
- 9 Write an algorithm to sort an array using selection sort.
- 10 Write an algorithm to sort an array using insertion sort.
- 11 Explain sequential search technique? Write an algorithm to search an element using sequential search.
- 12 Write an algorithm to search an element using binary search.



SARDAR PATEL UNIVERSITY
S. Y. B. Sc. (Sem. – IV)
Part – I : Advanced ‘C’ and Introduction to Data Structure
Practical Definition List

1.	Write a program to input and print the value and address of an integer variable using pointer.
2.	Write a program to input two numbers. Perform addition operation on them and print final answer using pointers.
3.	Write a program to input two numbers. Find out maximum number from them using pointer. Display appropriate message.
4.	Write a program to input two numbers and interchange them using pointers.
5.	Write a program to input one number and reverse it.
6.	Write a program to input one array and print it.
7.	Write a program to input one array and print it in reverse order.
8.	Write a program to input one array and add all elements of it and print the sum.
9.	Write a program to input one array and add even number and odd number and print the sum of both.
10.	Write a program to input two numbers add them and subtract them using function.
11.	Write a program to pass entire array to a function maximum (). Find maximum element from array and display it in main function.
12.	Write a program to pass entire array to a function maximum () using pointer. Find maximum element from array and display it in main function.
13.	Write a program to read an array of ‘n’ element (use malloc function to declare array dynamic allocation)
14.	Write a program to read and write library data like title of book, author, pages and price using structure.
15.	Write a program to define structure simple interest, which containing three members principal amount, rate of interest and no. of years. Read the value from user and calculate simple interest
16.	Write a program to create structure calculate. Declare two members of this structure (x and y). Find the addition, subtraction, multiplication and division of the member (use concept of initialization of structure)
17.	Write a program that read roll no, name and marks of 3 subjects of a student using structure and print the result for same student with total marks and percentage

18.	Write a program that read rollno, name and marks of 9 subjects of a student using array within structure and print the result for same student with total marks and percentage.
19.	Write a program that read roll no, name and marks of 9 subjects of n students using array of structure and array within structure. Print the result for each student with total marks and percentage.
20.	Write a program to read 10 names and their telephone number. Search a given telephone number, if it is available in the list then display its name and telephone number.
21.	Write a C program to read student information like roll number, name and marks of 3 subjects using structure. Pass structure as an argument to user defined function and print the data in user defined function.
22.	Write a program to read student number, name and city from the keyboard. Write these data into a file "stud1.txt".
23.	Write a program to read student number, name and city from the keyboard. Write these data into a file "stud1.txt". Read "stud1.txt" and display information on the screen.
24.	Write a program to read name of file from user and check whether it is present or not in disk. Give an appropriate message.
25.	Write a program to read name of the file from user and check whether it is present or not in disk. If present then display its contents on the screen, otherwise display an appropriate message.
26.	Write a program to count total number of character from given file.
27.	Write a program to find total number of uppercase letter, lowercase letter, blank space and special character from given file.
28.	Write a program to merge two files into a single file. i.e. Append content of one file at the end of another file.
29.	Handling of files with Mixed data type using fscanf and fprintf.
30.	Write a menu driven program for stack and perform following operations. Input integer numbers as a stack element. (1) Push (Insert element) (2) Pop (Delete element) (3) Peep (4) Change (5) Display (6) Exit
31.	Write a menu driven program for simple queue with size 10 and perform following operations. Input integer numbers as a queue element. (1) Insert Element (2) Delete Element (3) Display (4) Exit Proper validation for underflow and overflow of queue must be done.